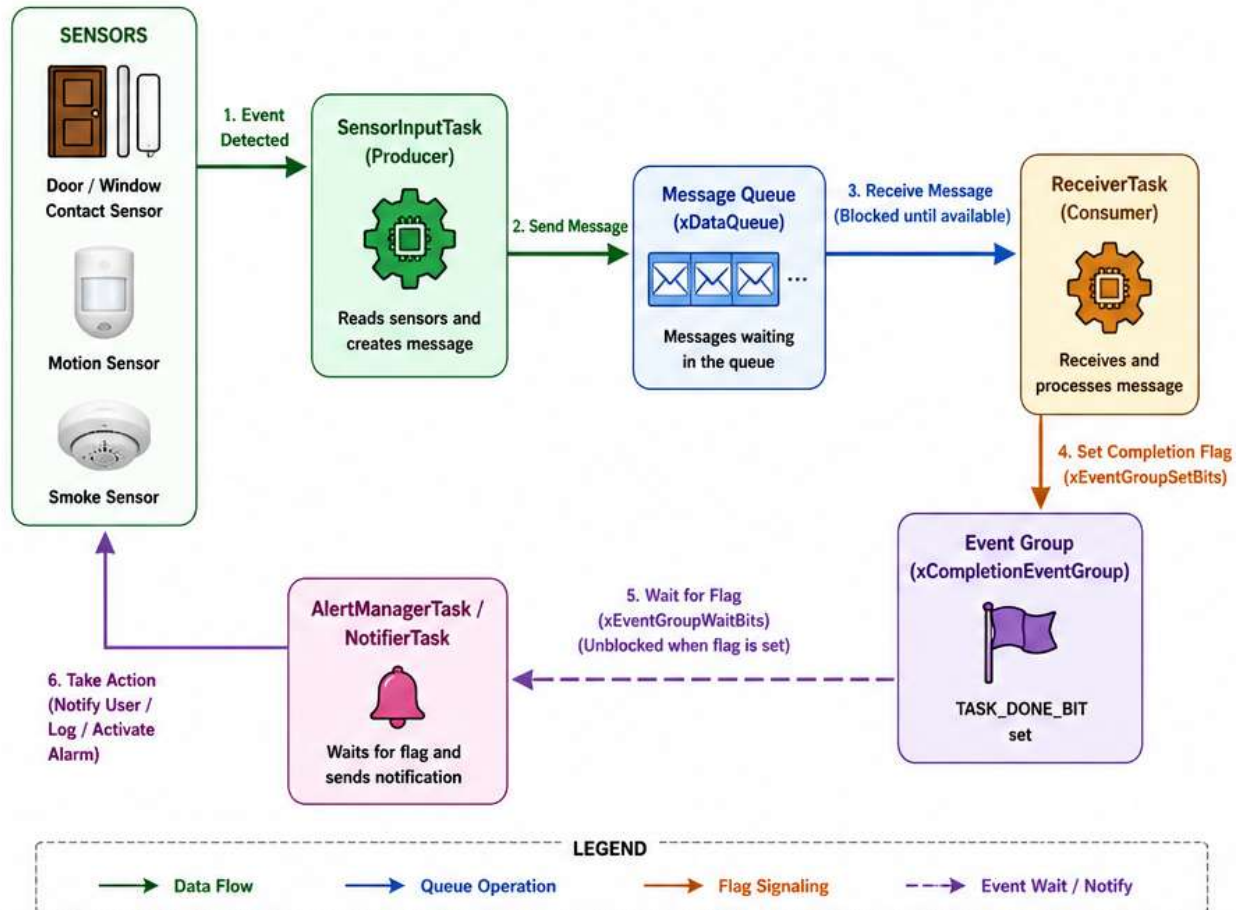


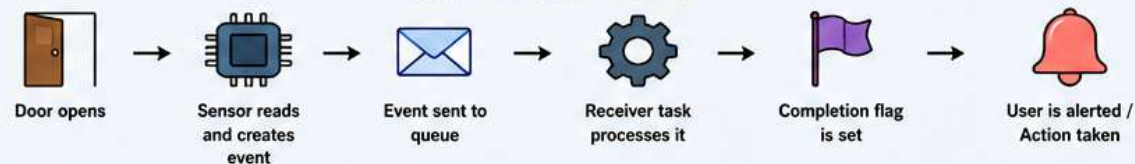
7. REAL-LIFE EXAMPLE

Example: A smart home security gateway.

SMART HOME SECURITY GATEWAY – FLOW OF OPERATIONS



WHAT HAPPENS IN REAL LIFE?



REAL-LIFE EXAMPLE:

Example: A smart home security gateway.

Consider a Wi-Fi enabled home security gateway built on an embedded RTOS. A “SensorInputTask” (producer) collects readings from door/window contact sensors and pushes each detected event (e.g., “door opened”) into a message queue (xDataQueue).

A “ReceiverTask” (the task described in the pseudocode) blocks on xQueueReceive(). When it gets a message, it processes the event — for example, it might log it to local storage, check whether it violates a security rule (e.g., door opened while system is armed), and take appropriate action.

After processing, it calls xEventGroupSetBits(xCompletionEventGroup, TASK_DONE_BIT) to signal completion. An “AlertManagerTask”, which previously called xEventGroupWaitBits() to wait for this flag, is immediately unblocked and can now send a push notification to the homeowner’s phone and activate a local alarm.

HOW THIS SOLVES THE PROBLEM:

- The queue ensures reliable, ordered delivery of sensor events without busy-waiting.
- The event flag provides efficient, one-to-many completion notification without requiring polling.
- The system remains responsive and power-efficient — ideal for a battery-backed embedded gateway.

BENEFITS IN REAL LIFE:

- Ensures no sensor event is missed.
- Keeps CPU usage low (tasks block instead of polling).
- Fast and reliable notification to other tasks.
- Scalable: more tasks can wait on the same event flag if needed.

QUESTION NO: 15

- Write pseudocode for a task receiving messages from a queue and signaling completion using event flags.

PROBLEM ANALYSIS:

The task acts as a consumer to receive messages from a queue and process them. After successful processing, it signals completion using an event flag so that other tasks can synchronize on it.

RTOS SELECTION:

FreeRTOS is chosen for this solution.

REQUIRED PRIMITIVES:

- Queue for inter-task communication (xQueueReceive)
- Event Group for signaling completion (xEventGroupSetBits)
- Blocking API to wait efficiently without wasting CPU cycles

DATA STRUCTURE:

- xDataQueue : QueueHandle_t
- xCompletionEventGroup : EventGroupHandle_t
- message_t : Data type for messages
- TASK_DONE_BIT : Event bit for completion

SYSTEM OVERVIEW



PSEUDOCODE (Contd...)

BEGIN

// SYSTEM INITIALIZATION

```
CREATE QUEUE xDataQueue WITH SIZE QUEUE_SIZE
CREATE EVENT GROUP xCompletionEventGroup
CREATE TASK ReceiverTask WITH PRIORITY N
START SCHEDULER
```

END

TASK: ReceiverTask

BEGIN

```
DECLARE message_t msg
DECLARE status return_status
INITIALIZE timeout_val = WAIT_TIME

WHILE TRUE
    RECEIVE MESSAGE FROM QUEUE
    status = xQueueReceive(xDataQueue, &msg, timeout_val)
    IF status == PASS THEN
        PROCESS THE RECEIVED MESSAGE
        ProcessMessage(msg)
        SIGNAL COMPLETION USING EVENT FLAG
        xEventGroupSetBits(xCompletionEventGroup, TASK_DONE_BIT)
    ELSE
        // Timeout occurred, no message received
        CONTINUE // Go back and wait again
    END IF
END WHILE
END
```

2

PSEUDOCODE (Contd...)

TASK: WaitingTask

BEGIN

```
DECLARE EventBits_t received_bits
DECLARE wait_flag = TASK_DONE_BIT
DECLARE wait_option = pdTRUE

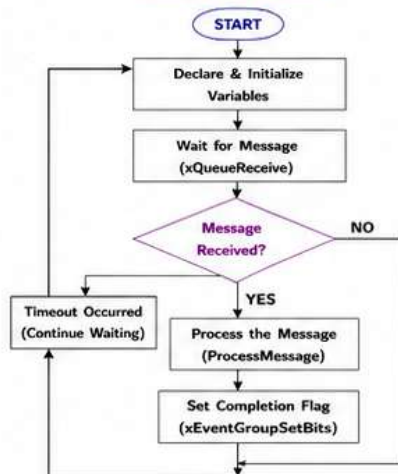
WHILE TRUE
    // WAIT UNTIL COMPLETION FLAG IS SET
    received_bits = xEventGroupWaitBits(xCompletionEventGroup,
                                        wait_flag, pdTRUE, pdFALSE, portMAX_DELAY)
    IF (received_bits & TASK_DONE_BIT) != 0 THEN
        // Completion received
        PerformNextOperation()
    END IF
END WHILE
END
```

QUEUE OPERATION SUMMARY

Operation	Performed By	Purpose
xQueueSend(xDataQueue)	Producer Task	Place message into queue
xQueueReceive(xDataQueue)	Receiver Task	Retrieve message from queue
xEventGroupSetBits()	Receiver Task	Signal completion
xEventGroupWaitBits()	Waiting Task(s)	Wait for completion signal

3

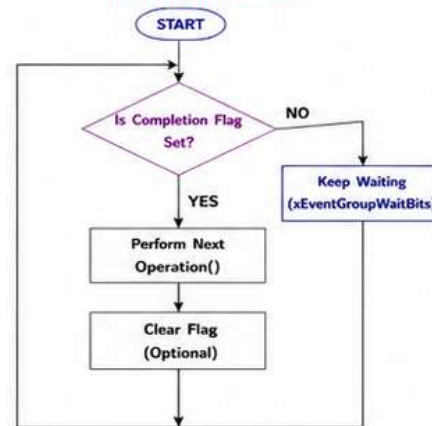
RECEIVER TASK FLOW



4

LOGICAL FLOW DIAGRAM

WAITING TASK FLOW



EXPLANATION:

The ReceiverTask waits for messages from the queue. When a message is received within the wait time, it is processed and the completion flag (TASK_DONE_BIT) is set in the event group. Other tasks waiting on this flag using xEventGroupWaitBits() will be notified and can proceed with their next operations.

KEY POINTS:

- Queues are used for data transfer between tasks.
- Event groups are used for status signaling and synchronization.
- The task blocks efficiently while waiting, avoiding CPU wastage.
- Multiple tasks can wait on the same event flag.

ASSUMPTIONS:

- Queue and event group are created during system initialization.
- Message processing is fast and non-blocking.
- WAIT_TIME is a predefined timeout value.
- Only one completion bit (TASK_DONE_BIT) is used for simplicity.
- No priority inversion issues occur.

5